



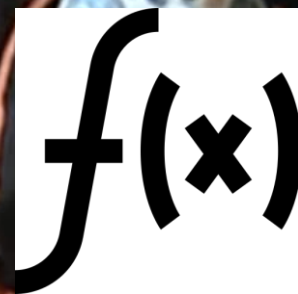
It's Your Life

with





Callback!?



학부모 호출

놔줄 테니까 부모님 모시고 와

```
function callbackLog(cb) {  
  console.log('==== callbackLog 함수 호출 ====');  
  cb();  
}  
  
function sayMuyaho() {  
  console.log('무야호~ ');  
}  
  
callbackLog(sayMuyaho);
```

매개 변수로 함수를 받기!



매개 변수로 받은 함수를 실행

```
[Running] node  
"c:\Users\student\Documents\Gitl  
==== callbackLog 함수 호출 ====  
무야호~
```



```
function callbackLog(cb) {  
  console.log('==== callbackLog 함수 호출 ====');  
  cb();  
}
```

```
function sayMuyaho() {  
  console.log('무야호~ ');  
}
```

```
function sayMyName() {  
  console.log('내 이름은? 이효석');  
}
```

```
callbackLog(sayMuyaho);  
callbackLog(sayMyName);
```

각기 다른 함수를 전달하여
해당 함수에게 공통 된 작업을 하고
해당 함수를 실행 가능!

[Running] node

"c:\Users\student\Documents\GitH

==== callbackLog 함수 호출 ====
무야호~

==== callbackLog 함수 호출 ====
내 이름은? 이효석



```
function callbackLog(cb) {  
  console.log('==== callbackLog 함수 호출 ====');  
  cb();  
}
```

```
callbackLog(function () {  
  console.log('아 함수 선언 귀찮아~~');  
});
```

매번 함수를 선언하고 전달 하는 방법이 아니라
그때 그때 상황에 맞게 함수를 정의해서 전달
가능!!

[Running] node

"c:\Users\student\Documents\GitH

==== callbackLog 함수 호출 ====

아 함수 선언 귀찮아~~



이거 어디서 많이 봤는데?

feat. 트리플H

```
btnEl.addEventListener('click', function () {  
    alert('TEST');  
});
```

```
btnEl.addEventListener('click', () => {  
    alert('TEST');  
});
```





실습, 간단한 콜백 함수 구현하기!

- 다음 장의 코드에서 youKnow() 함수를 완성하여 아래의 결과 화면과 동일한 결과를 만들어 주세요.
- xxx 는 본인의 이름과 본인이 먹을 저녁을 넣어 주시면 됩니다!
- 마지막 youKnow 함수는 익명 함수를 이용해서 함수를 직접 전달하는 방식으로 구현해 주세요

```
[Running] node "c:\Users\student\Doc
You know ~
엄.... 제 이름은 xxx 입니다
You know ~
내가 이따 먹을 저녁 메뉴는 xxx 입니다
You know ~
내일 아침은 xxx 를 먹겠어요
```

// youKnow() 함수를 작성해 주세요

```
function sayMyName() {  
  console.log('엄.... 제 이름은 xxx 입니다');  
}
```

```
function sayMyDinner() {  
  console.log('내가 이따 먹을 저녁 메뉴는 xxx 입니다');  
}
```

```
youKnow(sayMyName);  
youKnow(sayMyDinner);  
youKnow(/* 여기를 구현해 주세요*/);
```





비동기 통신

동기 / 비동기 프로그래밍



- 동기

- 하나의 요청이 완료가 되면 그 후에 다음 요청을 실행

- 비동기

- 하나의 요청이 완료가 되기 전에 다음 요청을 실행



SYNCHRONOUS VS ASYNCHRONOUS

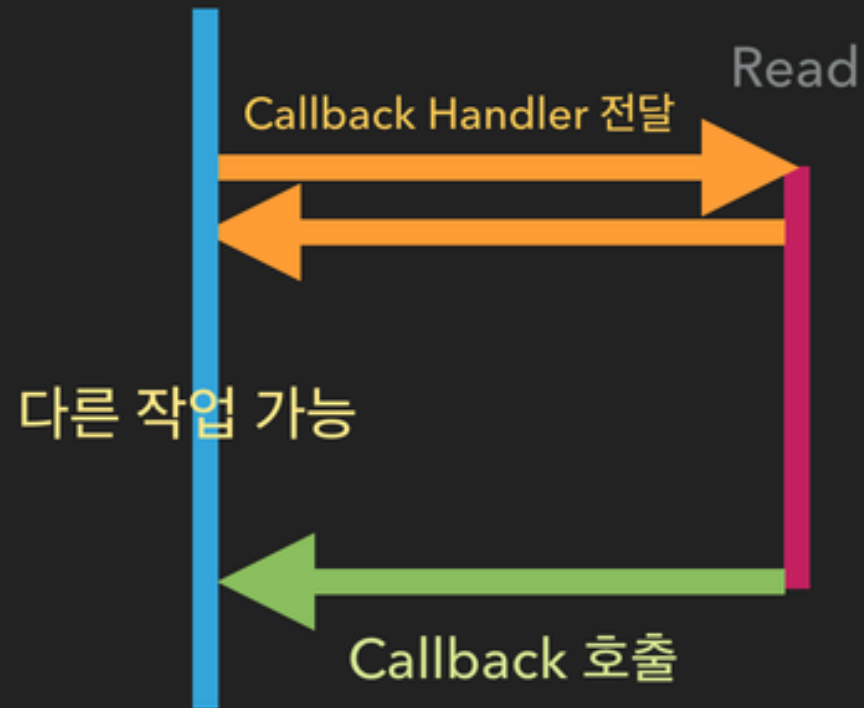
Sync 작업 완료 후 리턴

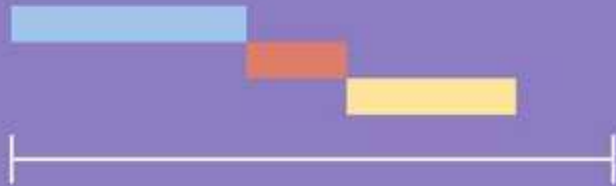
App Thread



Async 바로 리턴 + Callback

App Thread





Synchronous



Asynchronous



(a) 동기



(b) 비동기

비동기 프로그래밍



- JS 는 통신의 결과를 기다리지 않고 다른 일을 처리할 수 있습니다
- 즉, 동시에 여러가지 일을 한꺼번에 수행할 수 있습니다!
- 따라서 Node.js 는 가볍고 빠른 서버를 구성할 수 있죠!
- 하지만 이러한 비동기적 특성이 항상 좋은 것은 아닙니다!

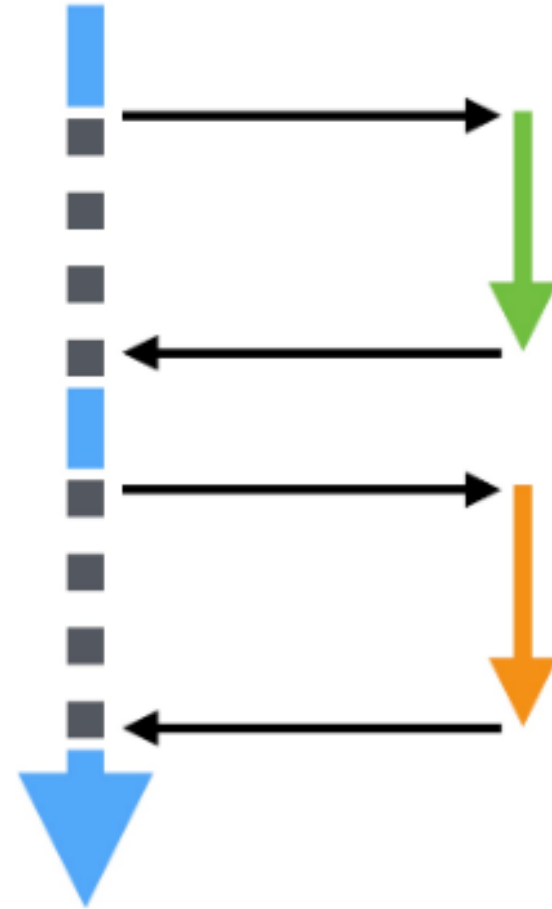
비동기 프로그래밍



- 서버(백엔드) 통신을 할 때에는 동기적(절차적)으로 문제를 해결해야합니다.
- 예를 들어서 로그인 요청을 보내고 서버에서 로그인 결과에 대한 결과 값을 받은 다음에 로그인 처리를 해야겠죠?
- 하지만 JS 는 비동기적 특성을 가지므로 + 아무리 인터넷이 빨라 졌어도 JS 의 처리 속도 보다는 느리기 때문에, 기존과 동일하게 코드를 작성하면 로그인 결과를 받기도 전에 JS 는 로그인 처리를 하려고 할 것 이고 이는 문제가 될 것입니다!



동기 (Sync)





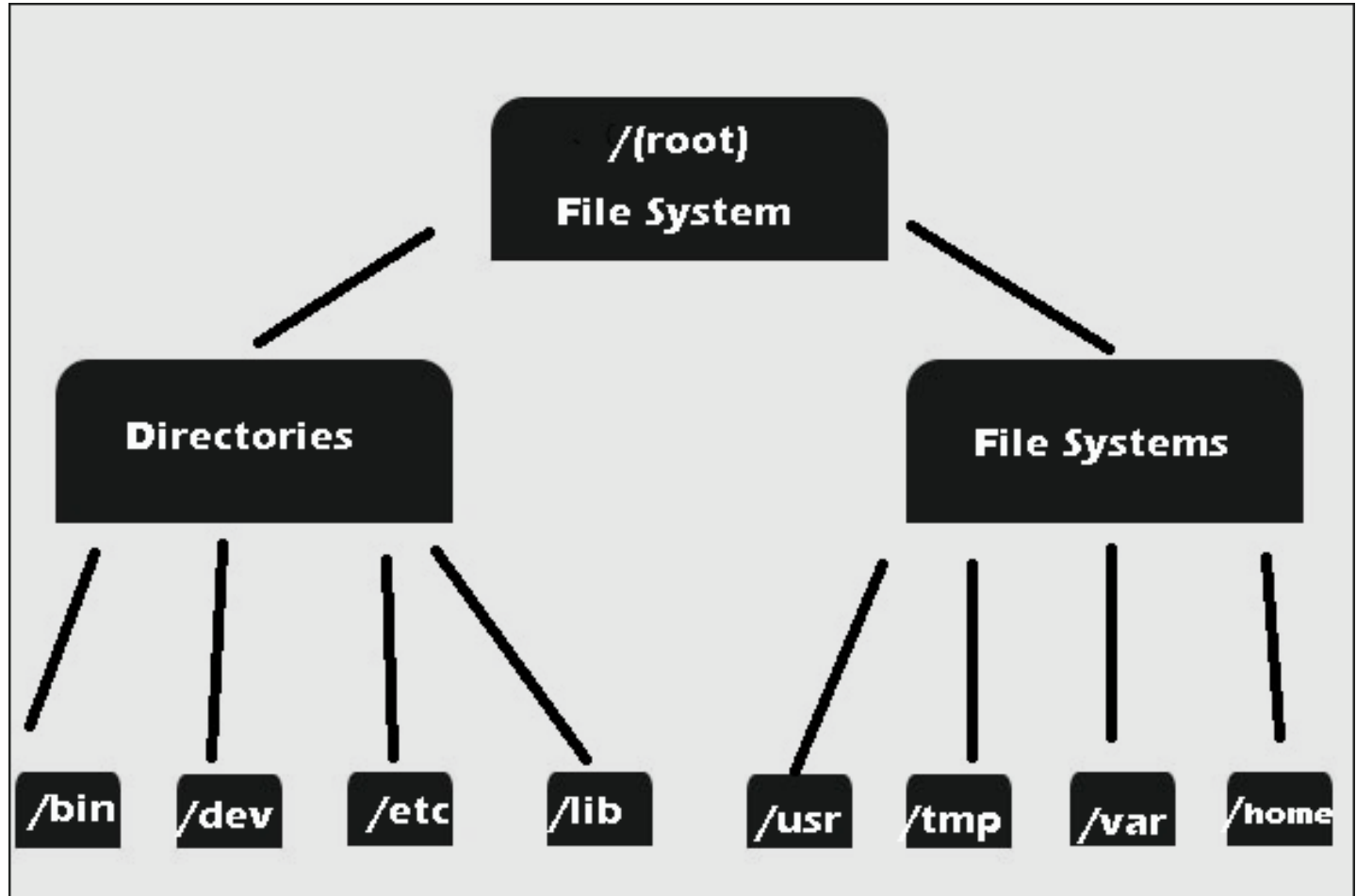
이론



현실



File-System on JS



File-system 은 Node 의 기본 모듈 입니다!



```
const fs = require('fs');
```

- 파일을 읽는 것도 시간이 필요한 작업이므로 서버 통신과 비슷합니다 → 따라서, 비동기적 처리가 필요하며 기본적으로 callback 을 지원합니다!
- fs.readFile('파일위치', '유니코드포맷', callback(err, data) {})
- err 은 파일 읽기가 잘 안되었을 때, Error 코드를 반환 합니다
- data 는 파일 읽기가 잘 되었을 때, 읽은 data 를 반환합니다.



파일 읽기!



```
const fs = require('fs');

fs.readFile('readme.txt', 'utf-8', function (err, data) {
  if (err) {
    console.log(err);
  } else {
    console.log(data);
  }
});
```

```
const fs = require('fs');

fs.readFile('readme.txt', 'utf-8', (err, data) => {
  if (err) {
    console.log(err);
  } else {
    console.log(data);
  }
});
```



File-system 으로

비동기적 특성

체험하기



File-system 과 비동기 프로그래밍

- 파일 시스템을 이용해서 비동기 프로그래밍 코드를 짜봅시다!
- JS의 특성으로 인해 각각 readFile 메소드를 동시에 비동기적으로 실행 시켜 봅시다!
- 동시에 readme.txt 파일을 읽어서 console.log 로 출력하는 코드를 작성해 봅시다!

```
const fs = require('fs');

fs.readFile('./readme.txt', (err, data) => {
  if (err) {
    throw err;
  }
  console.log('1번', data.toString());
});

fs.readFile('readme.txt', (err, data) => {
  if (err) {
    throw err;
  }
  console.log('2번', data.toString());
});

fs.readFile('readme.txt', (err, data) => {
  if (err) {
    throw err;
  }
  console.log('3번', data.toString());
});

fs.readFile('readme.txt', (err, data) => {
  if (err) {
    throw err;
  }
  console.log('4번', data.toString());
});
```



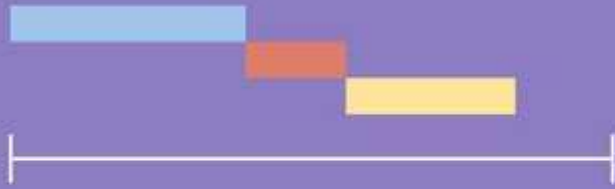


File-system 과 비동기 프로그래밍

- 과연 결과는 어떻게 될까요!?

```
[Running] node "d:\git\KB_  
1번 readme 텍스트 입니다!  
2번 readme 텍스트 입니다!  
3번 readme 텍스트 입니다!  
4번 readme 텍스트 입니다!
```

```
[Running] node "d:\git\K  
2번 readme 텍스트 입니다!  
1번 readme 텍스트 입니다!  
3번 readme 텍스트 입니다!  
4번 readme 텍스트 입니다!
```



Synchronous



Asynchronous



File-system 과 비동기 프로그래밍

- File 을 읽는 것은 JS가 각각의 Thread 에 실어서 처리하기 때문에 각각의 Thread 상황에 따라 file 읽는 속도가 다르게 됩니다.
- 따라서, 꼭 1, 2, 3, 4 로 실행 된다는 보장이 없죠!
- 그럼 1, 2, 3, 4 순서로 실행 시키려면 어찌하면 될까요?



Callback!?

구원자, Callback!



- 위와 같은 비동기 프로그래밍의 문제를 해결하기 위해서 JS 에서 지원하는 방식이 바로 Callback 방식 입니다~!
- Callback 함수는 특정 함수의 작업이 종료 되어야만 다른 함수를 호출 하기 때문에 사용자가 원하는대로 절차적 프로그래밍이 가능해 집니다!
- 그럼 일단 간단한 코드로 확인해 보시죠!

```
function generateRandomNumber() {  
  console.log('숫자 생성 중...');  
  
  setTimeout(() => {  
    const randomNumber = Math.floor(Math.random() * 10) + 1;  
    console.log(`생성된 숫자: ${randomNumber}`);  
  
    if (randomNumber >= 5) {  
      console.log('결과: 통과');  
    } else {  
      console.log('결과: 탈락');  
    }  
  }, 1000);  
  
  console.log('결과 확인 완료!');  
}  
  
generateRandomNumber();
```

setTimeout 함수와
아래의 console.log 가
비동기적으로 실행이 됩니다



```
function generateRandomNumber(callback) {  
  console.log('숫자 생성 중...');  
  
  setTimeout(() => {  
    const randomNumber = Math.floor(Math.random() * 10) + 1;  
    console.log(`생성된 숫자: ${randomNumber}`);  
  
    const result = randomNumber >= 5 ? '통과' : '탈락';  
  
    // 결과를 콜백 함수로 전달  
    callback(randomNumber, result);  
  }, 1000);  
}
```

setTimeout의 함수 종료 시점에
Callback 함수를 호출 합니다!

```
generateRandomNumber((number, result) => {  
  console.log(`결과: ${result}`);  
  console.log('결과 확인 완료!');  
});
```

전달 받은 값을 출력하여
비동기적 처리 완료



File with Callback



```
const fs = require('fs');

fs.readFile('readme.txt', (err, data) => {
  if (err) {
    throw err;
  }
  console.log('1번', data.toString());
  fs.readFile('readme.txt', (err, data) => {
    if (err) {
      throw err;
    }
    console.log('2번', data.toString());
  });
});
```

2번째 readFile 메서드를
1번째 readFile 메서드 내부에 작성!

이런 방식을 활용하여
순서를 강제하기!

[Running] node

1번 안녕하세요

2번 안녕하세요

Callback 으로 작성 된 코드는
아무리 많이 실행을 해도
실행 순서가 지켜지는 것을 볼 수 있습니다!





Callback

Hell?

콜백 지옥? 그게 뭐죠?



```
1  var floppy = require('floppy');
2
3
4  floppy.load('disk1', function (data1) {
5    floppy.prompt('Please insert disk 2', function() {
6      floppy.load('disk2', function (data2) {
7        floppy.prompt('Please insert disk 3', function() {
8          floppy.load('disk3', function (data3) {
9            floppy.prompt('Please insert disk 4', function() {
10             floppy.load('disk4', function (data4) {
11               floppy.prompt('Please insert disk 5', function() {
12                 floppy.load('disk5', function (data5) {
13                   floppy.prompt('Please insert disk 6', function() {
14                     floppy.load('disk6', function (data6) {
15                       //if no disk 6, then error
16                     });
17                   });
18                 });
19             });
20           });
21         });
22       });
23     });
24   });
25 });
26
27
```



```
foo(() => {
  bar(() => {
    baz(() => {
      quux(() => {
        quuz(() => {
          corge(() => {
            grault(() => {
              run();
            }).bind(this);
          }).bind(this);
        }).bind(this);
      }).bind(this);
    }).bind(this);
  }).bind(this);
}).bind(this);
```



```
function callbackHellFunc(cb) {  
  cb();  
}
```

```
callbackHellFunc(function () {  
  console.log(`1 번째 콜백 호출`);  
  callbackHellFunc(function () {  
    console.log(`2 번째 콜백 호출`);  
    callbackHellFunc(function () {  
      console.log(`3 번째 콜백 호출`);  
      callbackHellFunc(() => {  
        console.log(`4 번째 콜백 호출`);  
        callbackHellFunc(() => {  
          console.log(`5 번째 콜백 호출`);  
          callbackHellFunc(() => {  
            console.log(`6 번째 콜백 호출`);  
            callbackHellFunc(() => {  
              console.log('그만해.... 제발....');  
            });  
          });  
        });  
      });  
    });  
  });  
});
```

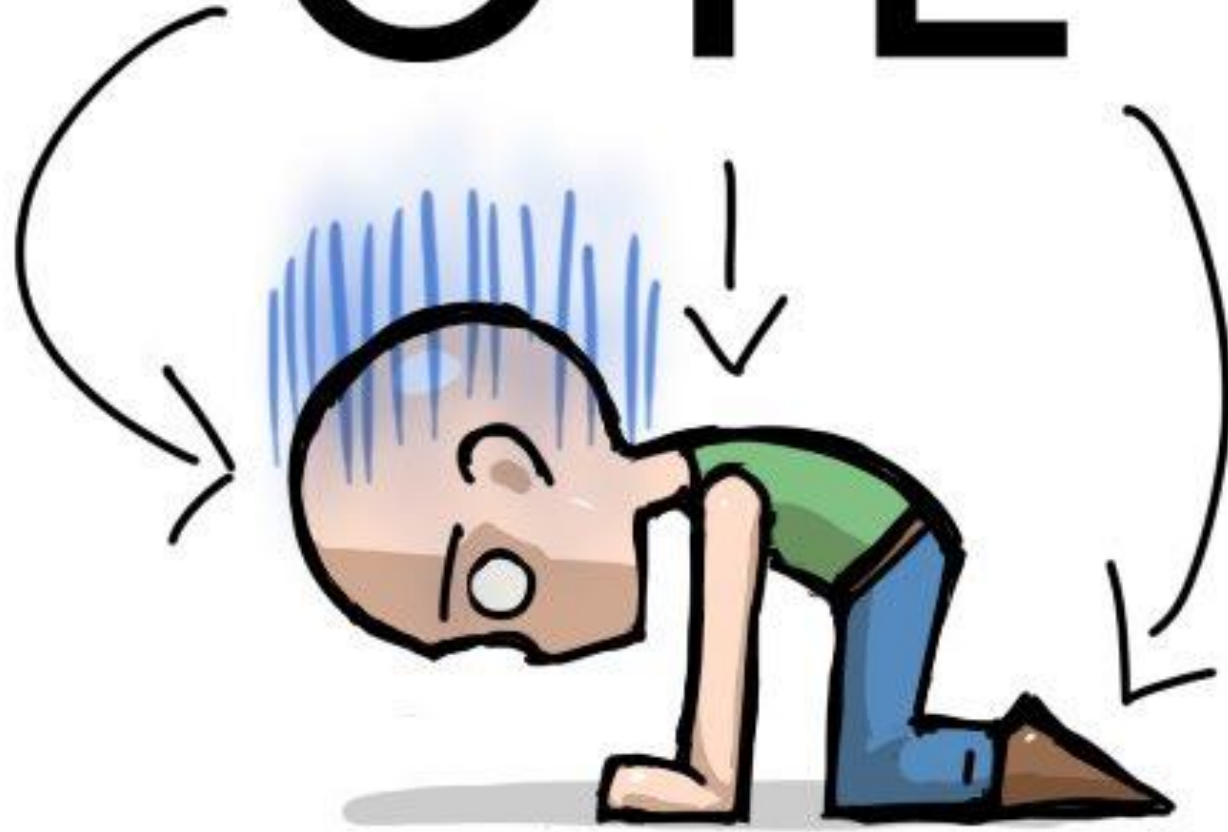


그 console.log
두 번째랑 네 번째를
좀 바꿔주겠니!?





OTL





HELL!!!!!!!!!!



Callback Hell

체험 해보기!

Callback



Hell?!



실습, 콜백 지옥 코드로 구현

- Fs 를 이용하여 파일 읽기 결과가 반드시 1번, 2번, 3번, 4번으로 나올 수 있게 코드를 구현하여 봅시다!
- 콜백 지옥을 사용하여 구현해야 합니다!

```
[Running] node "d:\git\K
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```



이상
콜백지옥 탈출 방법에
대해 알아 봤습니다!





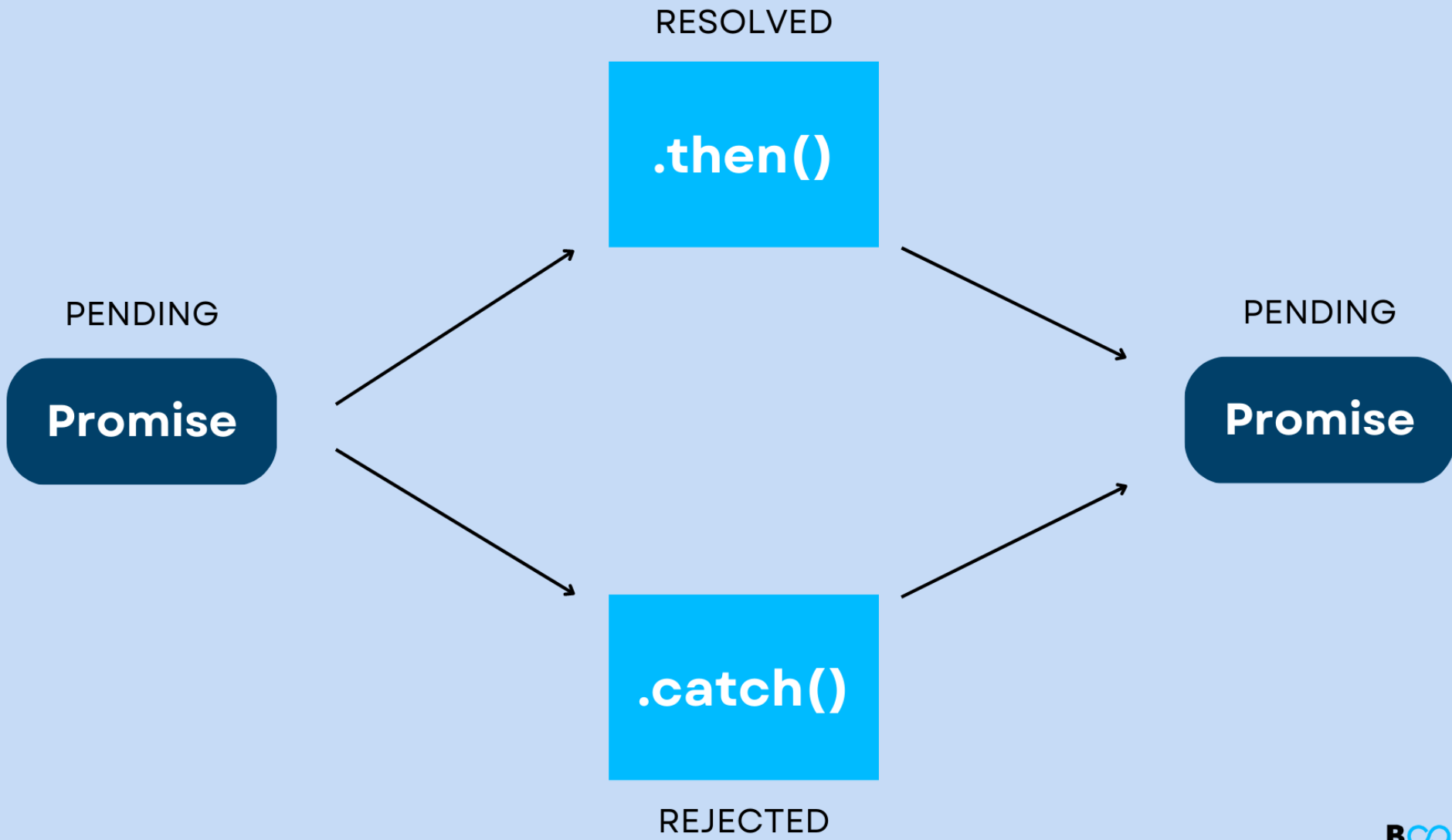
Promise!





Promise!

- Promise 는 Callback 을 대체하기 위해서 나온 개념입니다
- 말 그대로 '약속', 특정 작업을 수행한 다음 작업이 완료 되면 해당 결과 값을 돌려 주겠다는 약속! 입니다
- 대신, 비동기적 처리를 위해서 특정 약속이 수행 중일 때 JS 가 해당 결과를 기다려주는 기능이 추가가 되었습니다!





Promise!

- Promise 는 생성자 입니다! 따라서 new 로 사용하죠!

```
const promise = new Promise(function(resolve, reject) {});
```

- resolve, reject 라는 2개의 콜백 함수를 받아서 사용합니다.
- promise 가 할당 되면 이 promise 는 resolve 또는 reject 함수가 callback 될 때 까지 무한 대기 합니다.
- Resolve 는 promise 가 정상적으로 이행 되었을 경우 사용하며, reject 는 반대의 경우에 사용합니다.

Promise!



- **resolve** 는 추후에 **then** 으로 받으며, **reject** 는 **catch** 로 받습니다!
- resolve, reject 콜백 함수의 경우는 데이터를 매개변수로 보낼 수 있습니다.
- Resolve, reject 가 사용되지 않으면 promise 는 해당 콜백이 나올 때 까지 **pending** 상태가 되어 기다립니다!



Promise

올때까지 여기서 기다릴거야



```
const promise = new Promise(function (resolve, reject) {  
  console.log('프로미스 시작!');  
  setTimeout(() => {  
    console.log('setTimeout 끝!');  
    resolve('프로미스로 비동기 구현 성공!');  
  }, 2000);  
});  
  
console.log(promise);  
  
promise.then(function (data) {  
  console.log(data);  
});
```

프로미스 시작!

Promise { <pending> }

setTimeout 끝!

프로미스로 비동기 구현 성공!

setTimeout 으로 2초 대기!
& resolve 가 나올 때 까지
Pending 상태로 대기



콜백 지옥을 Promise 로 변경 하기

- `fs.promises` 를 사용해서 콜백 지옥을 promise 코드로 변경해 봅시다
- `fs.promises` 해당 메소드의 판단 여부를 스스로 판단 후 →
- 파일 읽기가 성공 하면 `resolve`
- 실패하면 `reject` 를 알아서 알아서 반환 합니다! → 고로 편리합니다!



```
const fs = require('fs').promises;
fs.readFile('./readme.txt')
  .then((data) => {
    console.log('1번', data.toString());
    return fs.readFile('./readme.txt');
  })
  .then((data) => {
    console.log('2번', data.toString());
    return fs.readFile('./readme.txt');
  })
  .then((data) => {
    console.log('3번', data.toString());
    return fs.readFile('./readme.txt');
  })
  .then((data) => {
    console.log('4번', data.toString());
  })
  .catch((err) => {
    throw err;
  });
```

```
[Running] node "c:\Users\s
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```

```
[Done] exited with code=0
```

```
[Running] node "c:\Users\s
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```

```
[Done] exited with code=0
```

```
[Running] node "c:\Users\s
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```

실습, 콜백 코드를 promise 로 구현하기



- 아래의 콜백 코드를 promise 로 구현하세요



```
function shouldIBuyLotto(callback) {  
  console.log('나 로또 사도 될까!?');  
  setTimeout(() => {  
    const rand = parseInt(Math.random() * 10);  
    console.log(`나온 숫자는 ${rand}`);  
  
    // rand 가 5 이상이면 로또 사자!  
    if (rand >= 5) {  
      callback('아싸! 로또 사자!');  
    } else {  
      callback('아... 망했어요...');  
    }  
  }, 3000);  
}  
  
function showResult(msg) {  
  console.log(msg);  
}  
  
shouldIBuyLotto(showResult);
```

```
[Running] node "c:\Users  
나 로또 사도 될까!?  
나온 숫자는 9  
아싸! 로또 사자!
```

```
[Done] exited with code=
```

```
[Running] node "c:\Users  
나 로또 사도 될까!?  
나온 숫자는 3  
아... 망했어요...
```



"그러나"

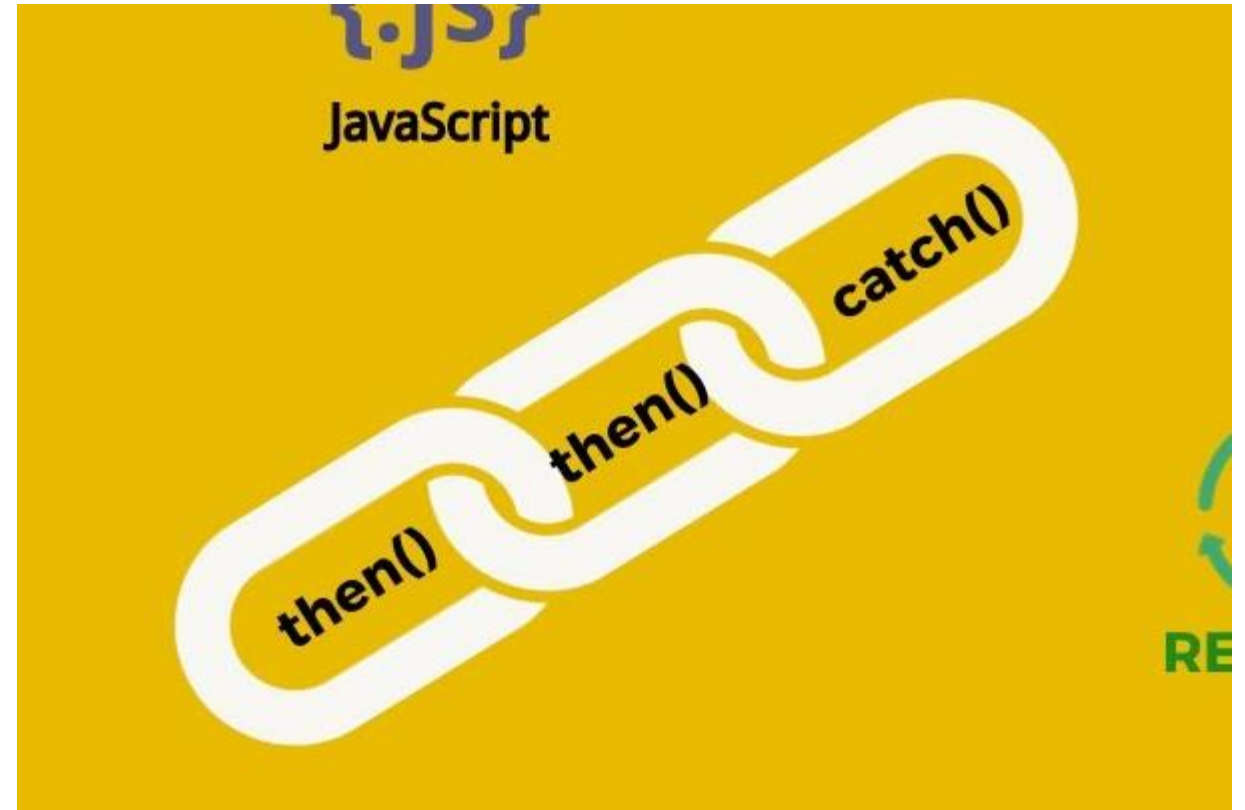
"그렇다고
볼 수 없다"

"자유가
될 수 없다"

"하지만"



```
Promise.resolve().  
  then(onFulfilled1).  
  then(onFulfilled2).  
  then(onFulfilled3).  
  catch(onRejected);
```

A diagram illustrating the execution flow of the provided JavaScript code. A green line starts from the right side of the first line, `Promise.resolve().`, and branches into three green arrows pointing to the right side of the subsequent `then()` calls. A red line starts from the left side of the first `then()` call, branches into two red arrows pointing to the left side of the second and third `then()` calls, and then continues as a red arrow pointing to the left side of the `catch()` call.



Async / Await



최신 기술인 Async, Await 도 적용!

- Function 앞에 `async` 를 붙이면 해당 함수는 항상 `Promise` 를 반환
- 즉, `async` 가 붙은 함수에서 `return` 을 쓰면 아래와 동일한 역할을 합니다.

```
async function f1() {  
  return 1;  
}  
  
async function f2() {  
  return Promise.resolve(1);  
}
```

- `Async` 가 붙은 함수 내부에는 `Await` 키워드 사용이 가능!
- `Await` 은 `promise` 가 결과(`resolve, reject`)를 가져다 줄 때 까지 기다립니다.

Async, Await 도 적용!



- 단, `async` 는 함수를 정의하는 상황에서 쓰이므로 함수 정의 후, 해당 함수를 외부에서 한번 사용해 줘야합니다!



Await





```
const promise = new Promise(function (resolve, reject) {  
  console.log('프로미스 시작!');  
  setTimeout(() => {  
    console.log('setTimeout 끝!');  
    resolve('프로미스로 비동기 구현 성공!');  
  }, 2000);  
});
```

```
promise.then(function (data) {  
  console.log(data);  
});
```

이 부분을 더 보기 편하게 바꿔볼 예정입니다!
그때 async/await 를 씁니다!



```
const promise = new Promise(function (resolve, reject) {  
  console.log('프로미스 시작!');  
  setTimeout(() => {  
    console.log('setTimeout 끝!');  
    resolve('프로미스로 비동기 구현 성공!');  
  }, 2000);  
});
```

```
async function asyncFunc() {  
  const result = await promise;  
  console.log(result);  
}
```

```
asyncFunc();
```

Await 에 의해 JS 가 저 위치에서 동작을 멈춥니다!

멈춘 동안 위에서 선언한 Promise 에서 시간이 흐르고
Resolve 가 반환 되는 순간!

Await 는 resolve 와 함께 사라져 버리고
Resolve 가 가져온 데이터만 리턴(=남게) 됩니다

프로미스 시작!

setTimeout 끝!

프로미스로 비동기 구현 성공!

Syntactic Sugar



{Syntactic Sugar}



- 같은 기능을 하지만 문법 상으로 더 편리하게 바꿔 주는 것을 Syntactic Sugar 라 부릅니다!
- Async, Await 는 promise 의 Syntactic Sugar 입니다!
- Promise 는 기존에 JS 코드 스타일과 다르기 때문에 promise 를 기존 코드 스타일로 사용할 수 있도록 만들어 준 것이 Async, Await 입니다!

```
promise.then(function (data) {  
  console.log(data);  
});
```

```
async function asyncFunc() {  
  const result = await promise;  
  console.log(result);  
}  
  
asyncFunc();
```

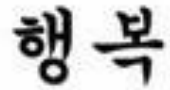


하시모토 칸나



황씨뒤또 짹마





그런데 이거 왜 쓰나요!?



```
const fs = require('fs').promises;
fs.readFile('./readme.txt')
  .then((data) => {
    console.log('1번', data.toString());
    return fs.readFile('./readme.txt');
  })
  .then((data) => {
    console.log('2번', data.toString());
    return fs.readFile('./readme.txt');
  })
  .then((data) => {
    console.log('3번', data.toString());
    return fs.readFile('./readme.txt');
  })
  .then((data) => {
    console.log('4번', data.toString());
  })
  .catch((err) => {
    throw err;
  });
```

```
const fs = require('fs').promises;

async function main() {
  let data = await fs.readFile('./readme.txt');
  console.log('1번', data.toString());

  data = await fs.readFile('./readme.txt');
  console.log('2번', data.toString());

  data = await fs.readFile('./readme.txt');
  console.log('3번', data.toString());

  data = await fs.readFile('./readme.txt');
  console.log('4번', data.toString());
}

main();
```



Async / Await 로

콜백지옥 탈출!



```
const fs = require('fs').promises;

async function main() {
  let data = await fs.readFile('./readme.txt');
  console.log('1번', data.toString());

  data = await fs.readFile('./readme.txt');
  console.log('2번', data.toString());

  data = await fs.readFile('./readme.txt');
  console.log('3번', data.toString());

  data = await fs.readFile('./readme.txt');
  console.log('4번', data.toString());
}

main();
```

```
[Running] node "c:\Users\st
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```

```
[Done] exited with code=0 i
```

```
[Running] node "c:\Users\st
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```

```
[Done] exited with code=0 i
```

```
[Running] node "c:\Users\st
1번 readme 텍스트 입니다!
2번 readme 텍스트 입니다!
3번 readme 텍스트 입니다!
4번 readme 텍스트 입니다!
```

실습, promise 코드를 async/await 로 변경



- 이전 실습에서 구현했던 promise 코드를 async / await 로 변경해 주세요
~! 😊



실제 서버 통신
체험 해보기!



Express

<http://expressjs.com/>



Back-End 서버의 기본!



Create → POST

Read → GET

Update → PUT

Delete → DELETE



GET localhost:4000/users ● POST lo

New Collection / localhost:4000/

GET



http://localhos

GET

Heade

POST

PUT

PATCH

DELETE

COPY

HEAD

OPTIONS

LINK

UNLINK

PURGE

LOCK

UNLOCK

PROPFIND

VIEW



HTTP Status Codes



1XX
INFORMATIONAL

2XX
SUCCESS

3XX
REDIRECTION

4XX
CLIENT ERROR

5XX
SERVER ERROR



HTTP Status

- 100 번째 : 정보 / 리퀘스트를 받고 처리 중
- 200 번째 : 성공 / 리퀘스트를 정상 처리
- 300 번째 : 리디렉션 / 처리 완료를 위해서는 추가 동작 필요
- 400 번째 : 클라이언트 에러 / 클라이언트에서 요청을 잘못 보냄
- 500 번째 : 서버 에러 / 리퀘스트는 잘 들어 갔지만 서버에서 처리를 못함



HTTP Status Codes

Level 200 (Success)

200 : OK

201 : Created

203 : Non-Authoritative
Information

204 : No Content

Level 400

400 : Bad Request

401 : Unauthorized

403 : Forbidden

404 : Not Found

409 : Conflict

Level 500

500 : Internal Server Error

503 : Service Unavailable

501 : Not Implemented

504 : Gateway Timeout

599 : Network timeout

502 : Bad Gateway

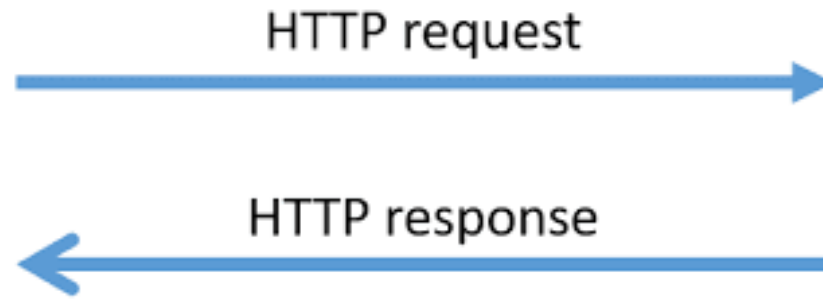


간단한

통신 경험하기!



Client



Server

Express 서버 구축



```
const express = require('express');
const cors = require('cors');

const PORT = 4000;

const app = express();
app.use(cors());

app.get('/request', (req, res) => {
  res.status(200).json('안녕하세요. 여기는 백엔드 입니다!');
});

app.get('/error', (req, res) => {
  res.status(500).json('비상!!! 에러 발생!!!!');
});

app.listen(PORT, () => {
  console.log(`데이터 통신 서버가 ${PORT}에서 작동 중입니다!`);
});
```



```
<!DOCTYPE html>
<html lang="en">
  <body>
    <h1 class="header">Hello, Protocol</h1>
    <button onclick="fetchData()">백엔드 통신 경험하기</button>
    <button onclick="errorHandling()">백엔드 에러 경험하기</button>
  </body>

  <script>
    const headerEl = document.querySelector('.header');
    const fetchData = () => {
      fetch('http://localhost:4000/request')
        .then((res) => {
          return res.json();
        })
        .then((data) => {
          console.log(data);
          headerEl.innerHTML = data;
        });
    };
  </script>

```

```
async function errorHandler() {  
  try {  
    const res = await fetch('http://localhost:4000/error');  
    const data = await res.json();  
    console.log(data);  
    headerEl.innerHTML = data;  
  } catch (err) {  
    console.log(err);  
  }  
}  
</script>  
</html>
```





Hello, Protocol

백엔드 통신 경험하기

백엔드 에러 경험하기

안녕하세요. 여기는 백엔드 입니다!

백엔드 통신 경험하기

백엔드 에러 경험하기

비상!!! 에러 발생!!!!

백엔드 통신 경험하기

백엔드 에러 경험하기

```
✖ ▶ GET http://localhost:4000/error 500 (Internal Server Error)
```